Total Detailed Overview of the Model

Hyderabad Metro Route Optimizer

How It Works • Each Component Explained • Worked Example

Team Members

Vanisha • Rachna • Akriti • Rohith Sai Gowd

Graph Data Model • Dijkstra & Yen's Algorithm

Six Route Objectives • Fare & Timetable Models

Door-to-Door Itinerary • Worked Example: Miyapur !' Nagole

1 What Is the Model?

The Hyderabad Metro Route Optimizer is a graph-based journey planning system that models the entire Hyderabad Metro network as a weighted graph and uses classical shortest-path algorithms to find optimal routes between any two stations. The model has six components that work together to produce a complete door-to-door journey plan.

The Six Components

1. **Graph Data Model**
   Builds a weighted graph from station and line data with vertices, edges, and interchange connections.
2. **Routing Engine**
   Dijkstra's algorithm and Yen's K-shortest paths find optimal routes under six different objectives.
3. **Fare Model**
   Estimates metro fares using HMRL's published distance-based slab structure (Rs. 10–60).
4. **Timetable Model**
   Simulated departure times with realistic operating hours (06:30–22:00) and headways (5–8 min).
5. **Walking Model**
   Estimates first/last-mile walking times at 5 km/h for door-to-door itinerary planning.
6. **Journey Planner**
   Combines all components into a complete timeline: walk !' wait !' ride !' tran

2 Network Overview & Schematic Map

The Hyderabad Metro operates 3 lines with 56 unique stations across 67.21 km of track. Three stations serve as interchanges where riders can transfer between lines without exiting the paid area.

Red Line
Miyapur – LB Nagar
27 stations • 29.21 km

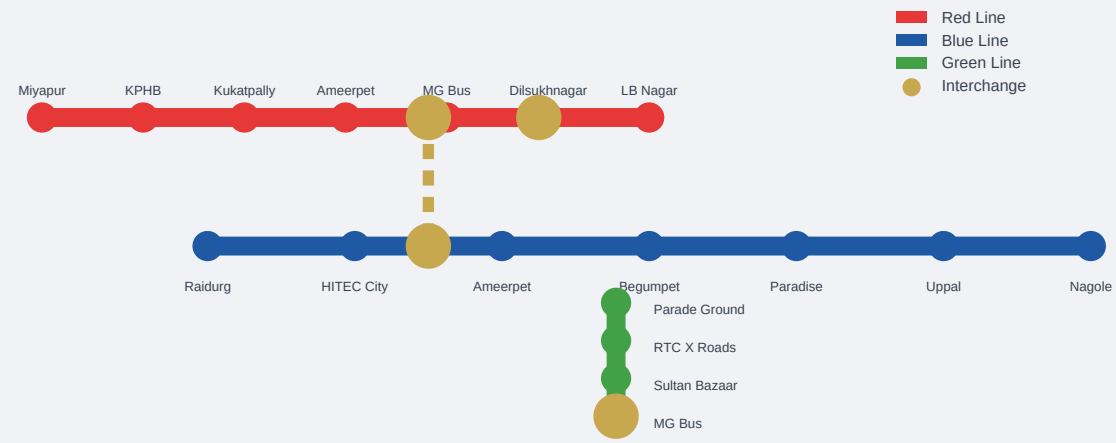
Blue Line
Raidurg – Nagole
23 stations • 27 km

Green Line
JBS Parade Ground – MGBS
9 stations • 11 km

Interchange Stations

- Ameerpet: Red Line – Blue Line (the busiest interchange, connecting corridors)
- Parade Ground: Blue Line – Green Line (connects Secunderabad area
- MG Bus Station (MGBS): Red Line – Green Line (major transit hub in

Schematic Network Map



3 Component 1: Graph Data Model

The foundation of the model is a weighted graph that represents the metro network. Each station on a line becomes a vertex, and connections between stations become weighted edges.

Vertices (59 total)

Each vertex is a (station, line) pair. A station served by N lines produces N vertices. For example, Ameerpet is served by both Red and Blue lines, so it has two vertices: "ameerpet:red" and "ameerpet:blue". Vertex IDs use the format stationId:lineId.

Edges (116 total)

Adjacent — **110 edges**

Connect consecutive stations on the same line. Weight: 1.1 km, 2.4 min. Bidirectional.

Interchange — **6 edges**

Connect same-station vertices across different lines. Weight: 0 km, 5 min (transfer

Walking — **0 edges**

Reserved for future out-of-station transfers (e.g., 300m walk between nearby stations).

Adjacency List

The graph is stored as an adjacency list: a map from each vertex ID to its list of neighbour entries. Each entry contains the target vertex and the edge (type, line, distance, time). This structure enables efficient traversal by Dijkstra's algorithm in O((V + E) log V) time.

```
ameerpet:red !' [sr_nagar:red (1.1km, 2.4min), punjaguttа:r
              ameerpet:blue (0km, 5min, interchange)]
ameerpet:blue !' [taruni_madhura_nagar:blue (1.1km, 2.4min)
               ameerpet:red (0km, 5min, interchange)]
```

4 Component 2: Routing Engine — Dijkstra’s Algorithm

Dijkstra’s algorithm finds the shortest path from a source vertex to all other vertices in a weighted graph. The model uses it with two weight modes: time (minimise travel time) and distance (minimise track kilometres).

Data Structures Used

- Adjacency List

Stores the graph; each vertex maps to its neighbours with edge weights.

- Min-Priority Queue

Binary heap that always returns the vertex with the smallest known distance.

- Distance Array

Map<vertexId, number> tracking the shortest known distance to each vertex.

- Predecessor Array

Map<vertexId, string|null> tracking the previous vertex on the shortest path.

- Settled Set

Set<vertexId> of vertices whose shortest distance is finalised.

Algorithm Steps

1. Initialise: Set source distance = 0, all others = ". Insert source into priorit
2. Extract minimum: Remove the vertex U with the smallest distance from the queue.
3. Settle U: Mark U as settled (its shortest distance is now final).
4. Relax neighbours: For each unvisited neighbour V of U, if dist[U] + weight(U,V) < dist[V], update dist[V] and set prev[V] = U. Insert V into the queue.
5. Repeat steps 2–4 until the queue is empty.
6. Reconstruct path: Walk backwards from destination to source using the predecessor array, then reverse.

Complexity

- Time: O((V + E) log V) with a binary heap priority queue
- Space: O(V + E) for the adjacency list and auxiliary arrays
- With 59 vertices and 116 edges, routes compute in milliseconds

5 Yen’s K-Shortest Paths & Six Route Objectives

Yen’s Algorithm

While Dijkstra finds the single shortest path, Yen’s algorithm finds the K shortest loopless paths between two vertices. It works by running Dijkstra for the first path, then iteratively "spurring" from each node on the previous best path, temporarily removing edges to force alternative routes. The model uses K=5 candidates for the cheapest and balanced objectives.

Six Route Objectives

Fastest	Dijkstra (time-weighted)	Minimises total travel time. Best for commuters who value speed.
Cheapest	Yen K=5 + fare slabs	Finds lowest fare. When all routes share a slab, notes this explicitly.
Fewest Interchanges	Lexicographic Dijkstra	Minimises line changes first, then time. Best for comfort.
Least Walking	Dijkstra (walking-weighted)	Minimises walking distance. Ready for future out-of-station transfers.
Shortest Distance	Dijkstra (distance-weighted)	Minimises total track kilometres. May differ from fastest.
Balanced	Yen K=5 + normalised scoring	Scores candidates on time, cost, transfers, walking with configurable weights.

6 Components 3–5: Fare, Timetable & Walking Models

Fare Model (Component 3)

Hyderabad Metro fares are distance-based with 11 slabs from Rs. 10 (0–2 km) to Rs. 60 (>30 km). The fare depends only on total metro distance, not the route taken. The model looks up the slab for the journey's total distance and returns the corresponding fare.

0–2 km	**Rs. 10**	2–4 km	**Rs. 15**
4–6 km	**Rs. 20**	6–8 km	**Rs. 25**
8–10 km	**Rs. 30**	10–14 km	**Rs. 35**
14–18 km	**Rs. 40**	18–22 km	**Rs. 45**
22–26 km	**Rs. 50**	26–30 km	**Rs. 55**
>30 km	**Rs. 60**		

Timetable Model (Component 4)

Since HMRL's GTFS feed is behind a request form, the model uses a SIMULATED timetable with realistic parameters: operating hours 06:30–22:00, headways of 5–8 minutes (peak vs off-peak). The nextDeparture() function finds the next train after a given time, computing waiting time for the itinerary timeline.

- Red Line: 6 min headway
- Blue Line: 5 min headway
- Green Line: 8 min headway
- All times labelled as SIMULATED in the UI

Walking Model (Component 5)

Estimates first/last-mile walking at 5 km/h average speed. Users can provide a walking time directly or an address (estimated via a deterministic heuristic producing 5–20 min). Interchange walking is modelled as a 5-minute penalty within the graph itself.

7 Component 6: Door-to-Door Journey Planner

The journey planner wraps the routing engine with access walking, timetable-based waiting, and exit walking to produce a complete timeline from leaving your start location to arriving at your destination.

Itinerary Timeline Steps

1	**Access Walk**	Walk from your start address to the boarding station
2	**Wait**	Wait for the next train on the first line (timetable-based)
3	**Board**	Board the train at the source station
4	**Ride**	Travel on the train through consecutive stations
5	**Interchange Walk**	Transfer between lines at an interchange station (5 min)
6	**Wait (transfer)**	Wait for the next train on the new line
7	**Board (transfer)**	Board the next line's train
8	**Ride (continued)**	Continue journey on the new line
9	**Exit Walk**	Walk from the alighting station to your final destination

Total Duration Calculation

Total = Access Walk + Waiting + Metro Ride + Interchange Penalties + Transfer Waiting + Exit Walk. All times are deterministic — no randomness. The result includes departure time, arrival time, total cost (metro fare + optional first/last-mile costs), and total walking distance.

8 Worked Example: Miyapur ⟩ Nagole

Let us trace a complete journey from Miyapur (Red Line terminal in the north-west) to Nagole (Blue Line terminal in the east). This is a cross-city journey that requires a transfer at Ameerpet, the busiest interchange station.

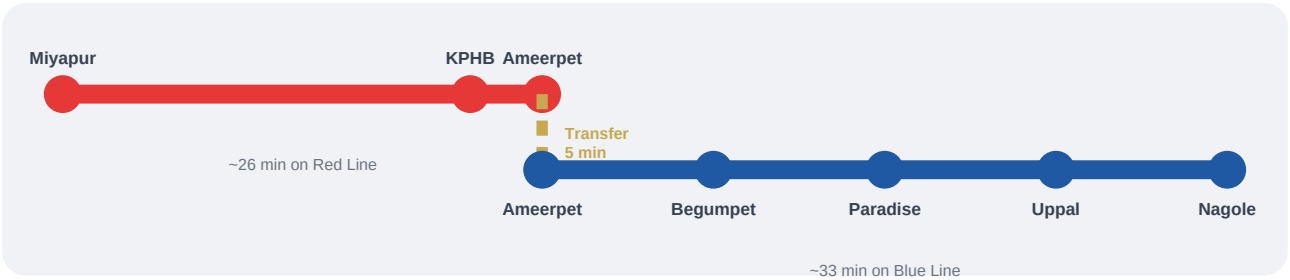
Source **Miyapur (Red Line)**	Destination **Nagole (Blue Line)**	Transfer at **Ameerpet**
Departure **09:00 AM**	Objective **Fastest**	Total Distance **~33 km**

Step 1: Build the Graph

The graph has 59 vertices and 116 edges. Miyapur maps to vertex "miyapur:red". Nagole maps to "nagole:blue". Since these are on different lines, the algorithm must find a path through an interchange vertex.

Step 2: Run Dijkstra (Time-Weighted)

Dijkstra starts at "miyapur:red" with distance 0. It explores the Red Line eastward, settling each station: JNTU College (2.4 min), KPHB Colony (4.8 min), Kukatpally (7.2 min)... until reaching Ameerpet on the Red Line (~26.4 min). At Ameerpet, it discovers the interchange edge to "ameerpet:blue" with a 5-minute transfer penalty. It continues on the Blue Line eastward: Begumpet, Paradise, Secunderabad East... until reaching "nagole:blue".



9 Worked Example: Results & Itinerary

Step 3: Path Reconstruction

Dijkstra reconstructs the path by walking backwards from "nagole:blue" through the predecessor array, then reversing. The full vertex path is:

```
miyapur:red → ... → ameerpet:red → ameerpet:blue (transfer) → ...
```

Step 4: Build Legs & Metrics

Total Travel Time	~64 min
Total Distance	~33 km
Interchanges	1 (at Ameerpet)
Fare	Rs. 60 (max slab)
Stops Traversed	~30 stations
Transfer Penalty	5 min

Step 5: Door-to-Door Itinerary Timeline

09:00	Leave your start location in Miyapur
09:05	Arrive at Miyapur station (5 min walk)
09:06	Board Red Line train (1 min wait)
09:06	Ride Red Line: Miyapur → Ameerpet (26 min, 10 stops)
09:32	Arrive at Ameerpet, transfer to Blue Line (5 min)
09:37	Wait for Blue Line train (3 min)
09:40	Board Blue Line at Ameerpet
09:40	Ride Blue Line: Ameerpet → Nagole (33 min, 13 stops)
10:13	Arrive at Nagole station
10:18	Walk to destination (5 min)
10:18	**TOTAL: 78 min door-to-door • Rs. 60 fare**

Note: Times are based on simulated estimates. Average edge durations do not guarantee a real train connection. The UI clearly labels all timetable data as SIMULATED.

10 How All Components Work Together

When a user searches for a route, the application follows this pipeline:

1. **User Input**
   User selects source, destination, departure time, and route objective on the Plan screen.
2. **Graph Lookup**
   Source and destination station IDs are resolved to graph vertices (station:line pairs).
3. **Routing Engine**
   Dijkstra or Yen's algorithm runs on the graph with the appropriate weight mode for the chosen objective.
4. **Metrics Computation**
   Total time, distance, interchanges, fare, and walking are recomputed from the path's actual edges.
5. **Itinerary Assembly**
   The journey planner wraps the route with access walk, timetable waiting, interchange timing, and exit walk.
6. **UI Display**
   Results shown as route cards with metrics, a schematic map, an itinerary timeline, and algorithm explanation.

Technology Stack

- React + TypeScript: UI components and type-safe data flow
- Vite: Fast development and production builds
- Tailwind CSS: Responsive, consistent styling
- Supabase: Backend (database, auth) — available but not required for routing
- All routing runs client-side — no server calls needed for route computation